

Barcelona Metro Lost-and-Found GNN — Overview

A short, jargon-light explanation of what the model does, how it was trained, and why it qualifies as a Graph Neural Network (GNN).

The problem

When a passenger loses an item on the Barcelona Metro:

1. Staff find it somewhere on the network (`found_at` station).
2. Days later the passenger comes to claim it. They also report (via questionnaire) where they think they lost it (`lost_at` station).

In the meantime we have to **store the item at some station on the network** — any station, not necessarily between the lost and found locations. The goal is to pick a storage station that minimizes the passenger's eventual claim trip.

The model predicts the passenger's **preferred pickup station** given the event details, and a separate decision rule turns that prediction into a storage placement.

What the model is trained on

A synthetic corpus of 50,000 lost-item events. Each event has:

Field	Meaning
<code>found_at</code>	Station where staff retrieved the item
<code>lost_at</code>	Station the claimant reports as the loss location
<code>item_type</code>	One of 13 categories (wallet, phone, umbrella, ...)
<code>hour</code> , <code>day</code>	When the loss occurred
<code>pickup</code>	Target: the station the claimant actually chose for retrieval

Split 80/10/10 into train / validation / test.

The events are generated by simulating three passenger profiles (commuter, tourist, leisure) walking from an origin station to a destination on the actual Barcelona metro graph. The item is "lost" somewhere on the route, carried a few stops further by the train (geometric distribution by item type), then "found" downstream. Pickup is the claimant's preferred station, sampled near their destination.

The KPI

Expected hops to pickup (`E[hops]`) — the metric we minimize and the one to lead with.

For every event, the decision rule picks a storage station `s` . We measure the metro-graph distance from `s` to the passenger's actual chosen pickup station, weighted by:

- **1.0** per stop along the same line
- **+1.0** per transfer (a line change adds a "virtual hop" of penalty)

This captures real passenger inconvenience: transfers are slow.

We also track `top-1` / `top-3` / `top-5` accuracy — was the true pickup station in the model's top-K predictions — but these are harsh binary metrics; `E[hops]` is the operational truth.

Performance vs non-GNN baselines

All numbers on the held-out test set (5,000 events). Lower `E[hops]` is better.

Model	E[hops]	top-1	top-5
GNN (full)	3.37	0.291	0.549
MLP (2-layer, hidden=64)	3.40	0.287	0.546
Logistic Regression	3.65	0.283	0.521
GNN without message passing	4.22	0.248	0.354
Random Forest (200 trees)	4.17	0.214	0.404
k-NN (k=25)	4.32	0.214	0.362
Baseline: <code>mode_lookup</code> (per-context empirical mode)	4.13	—	—
Baseline: <code>same_station</code> (predict pickup = found_at)	4.21	—	—

On in-distribution data the GNN beats every other model, but the margin over a plain MLP is small (just 0.03 hops). The honest claim from this table alone is "the GNN is competitive"; the stronger argument lives in the cold-start table below.

Where the GNN really earns its keep — sparse-data stations

Cold-start test holds out 15% of stations (24 of 157) entirely from training. No event whose `found_at` or `lost_at` touches those stations enters the graph or the training loop. We then evaluate every model on events that *do* touch those held-out stations — a stress test for sparse / unseen / long-tail stations.

Model	E[hops] (cold)	top-1	Δ vs GNN
GNN (full)	4.14	0.196	—
GNN without message passing	4.78	0.136	+0.64
Baseline: <code>same_station</code>	4.81	—	+0.67
Logistic Regression	5.03	0.125	+0.89
Baseline: <code>mode_lookup</code>	5.03	—	+0.89
MLP	5.54	0.129	+1.40

The inversion of fortune. The MLP is tied with the GNN on warm data (3.40 vs 3.37) but becomes the *worst* model on cold-start (5.54), behind every classical baseline. The GNN holds at 4.14. Going from warm to cold:

	Warm	Cold	Degradation
GNN (full)	3.37	4.14	+23%
MLP	3.40	5.54	+63%

Why this happens. The MLP and Logistic Regression encode stations as one-hot IDs. During training they never see the held-out one-hot positions activated, so the weights at those positions remain at their initialization. At test time those dimensions carry no useful signal. The GNN builds a station's

representation from (a) hand-crafted features (line membership, transfer flag, degree) AND (b) the representations of its metro neighbors via message passing — so unseen stations inherit predictive structure from observed ones nearby.

The takeaway for TMB. The GNN's value isn't a marginal accuracy bump on busy hub stations — it's robustness on the long tail:

- **Quiet stations with few events:** inherit predictions from busier neighbors instead of needing their own data.
- **Newly opened lines or stations** (L9 extensions, future expansions): can be predicted on day one based on their topological context, without waiting months for events to accumulate.
- **Distribution drift over time:** neighbor-based representations adapt more gracefully than per-station memorized weights.

The 1.40-hop cold-start advantage over MLP is, mechanically, the value of the graph-structured inductive bias.

What makes it a GNN

A GNN is defined by three things:

1. **A graph.** Our graph has two node types — **station** nodes (157, the Barcelona metro stops) and **context** nodes (104, one per item-type × time-bucket combination). Edges include physical metro adjacency, plus learned edges connecting stations to the contexts they appeared in.
2. **Neighborhood aggregation.** Each node updates its internal representation by summing messages from its neighbors. Concretely, each station's representation becomes a function of its line membership, transfer status, *and* what its metro neighbors look like.
3. **Stacked layers.** Two such update steps stack, so each station's final representation depends on its 2-hop neighborhood on the graph.

This architecture is a **heterogeneous relational GNN** in the R-GCN family (Schlichtkrull et al., 2018), with a **DistMult-style** bilinear scoring head (Yang et al., 2014). Heterogeneous means different edge types get different message functions; relational means the model treats the picked-up / found-at / lost-at / metro relations as distinct.

A concrete example: Jaume I

Take **Jaume I** (a station on line L4, between Barceloneta and Urquinaona). When the GNN updates Jaume I's representation in one layer it does:

1. **Self:** keep Jaume I's own features — line membership = L4, transfer flag = no, normalized degree.
2. **Metro neighbors:** sum the current representations of Barceloneta and Urquinaona, then transform with a learned weight matrix `W_metro`.
3. **Context neighbors:** every context (item × time-bucket) that had an event at Jaume I in training is summed in, transformed by separate weight matrices for the `found_at`, `lost_at`, and `picked_up` relations.
4. **Combine:** all four messages are summed, passed through LayerNorm → ReLU → dropout, and added as a residual to the previous representation.

After **layer 1**, Jaume I's representation incorporates Barceloneta and Urquinaona. After **layer 2** (the second stacked update), it incorporates *their* neighbors too — Drassanes and Passeig de Gràcia. So its final representation encodes a 2-hop window of the L4 corridor, plus the historical contexts

associated with any of those four stations. The same computation happens for every station in parallel, with the same shared weight matrices — just as a CNN applies the same filter at every pixel.

How this helps storage prediction

When a new event arrives — say, "*phone lost at Jaume I, found at Barceloneta, 18:30 on a Tuesday*" — the model assembles a query from three GNN-updated representations:

1. The **context** embedding for (phone, 18-24, weekday) .
2. The **found_at** station representation: Jaume I's vector, which already encodes its L4 corridor and observed patterns.
3. The **lost_at** station representation: the reported loss station's vector, similarly contextualised.

These three vectors are concatenated, passed through a small MLP to form a query `q`, and then `q` is scored against the GNN-updated representation of every station via a bilinear function. The output is a probability distribution over all 157 stations — the model's belief about where the passenger will pick up.

The decision rule then picks the storage station that minimises the expected metro-distance from this distribution. Because Jaume I's vector already encodes the L4 corridor structure, the predicted pickup distribution naturally concentrates probability mass along L4 — exactly where a tourist who lost their phone at Jaume I is most likely to return to claim it. The chosen storage station is then a station on that corridor that minimises expected hops.

This is the mechanism that powers the cold-start advantage. Even if Jaume I had zero training events, its representation would still encode the L4 corridor because its neighbors *do* have data. A model without message passing would treat Jaume I as an opaque ID, fail to relate it to L4 or its neighbors, and produce a near-marginal prediction — which is exactly what the MLP does on cold-start (5.54 hops vs the GNN's 4.14).

Why this is a real GNN, not a glorified lookup

We removed all message passing (keeping only the embeddings and the scoring head). The model dropped from 3.37 to 4.22 expected hops — *below* both non-GNN baselines. The GNN's performance lift over baselines comes entirely from the aggregation-and-transform mechanism described above.

References

- **R-GCN** — Schlichtkrull et al., *Modeling Relational Data with Graph Convolutional Networks* (2018). The blueprint for heterogeneous-relation GNNs we used.
- **GraphSAGE** — Hamilton et al., *Inductive Representation Learning on Large Graphs* (2017). Foundational for inductive GNNs (handling unseen nodes via neighborhood aggregation).
- **DistMult** — Yang et al., *Embedding Entities and Relations for Learning and Inference in Knowledge Bases* (2014). The bilinear scoring head we use for link prediction.

Quick file map

File	Purpose
<code>synth.py</code>	Generates the synthetic event corpus
<code>graph_build.py</code>	Assembles the heterogeneous graph from events
<code>model.py</code>	The HetGNN architecture

File	Purpose
<code>train.py</code>	Training loop
<code>decision.py</code>	Converts predicted pickup distribution into a storage choice
<code>ablate.py</code>	Ablation runner (used to verify GNN-ness)
<code>coldstart.py</code>	Cold-start station inductive evaluation
<code>ml_baselines.py</code>	Non-GNN ML baselines (MLP, LogReg, RF, kNN)
<code>ml_baselines_coldstart.py</code>	Non-GNN baselines on the cold-start split
<code>sweep_lambda.py</code>	Movement-cost tradeoff sweep